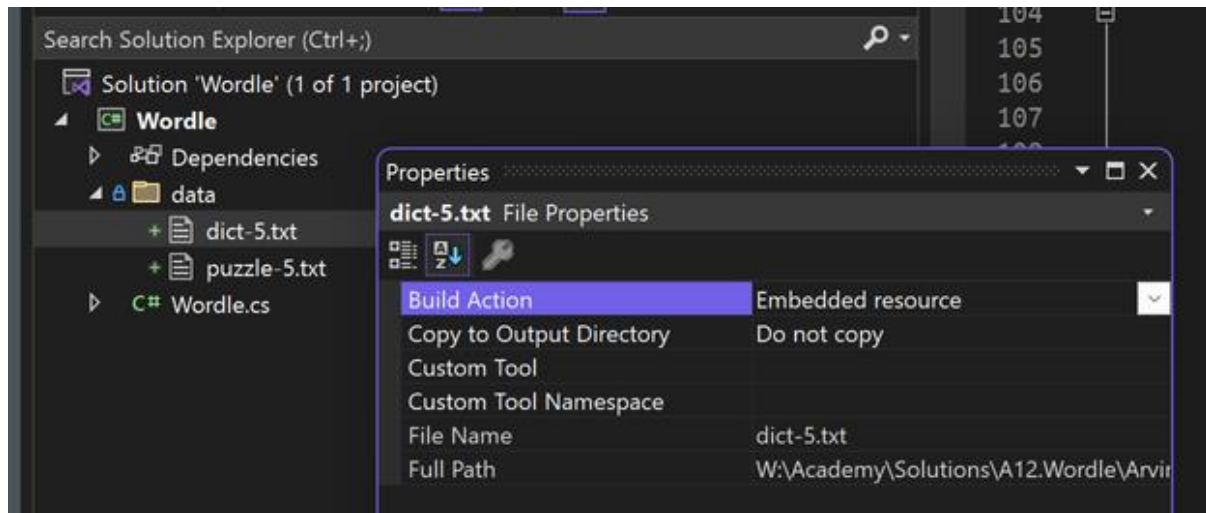


Background:

The attached ZIP file contains the reference solution for the Wordle assignment. An interesting technique to note is the way the data files (dict-5.txt and puzzle-5.txt) are being embedded right into the binary executable. To do this, add the files into the project, then right-click them and set the **Build action** to **Embedded resource**.



Then, you can read these 'embedded resources' by using the `Assembly.GetManifestResourceStream` function in .Net (see the **LoadStrings** routine in my code to understand how). This now makes the executable very robust, since it no longer depends on a text file being present in c:/etc or in some other hard-coded location.

Assignment:

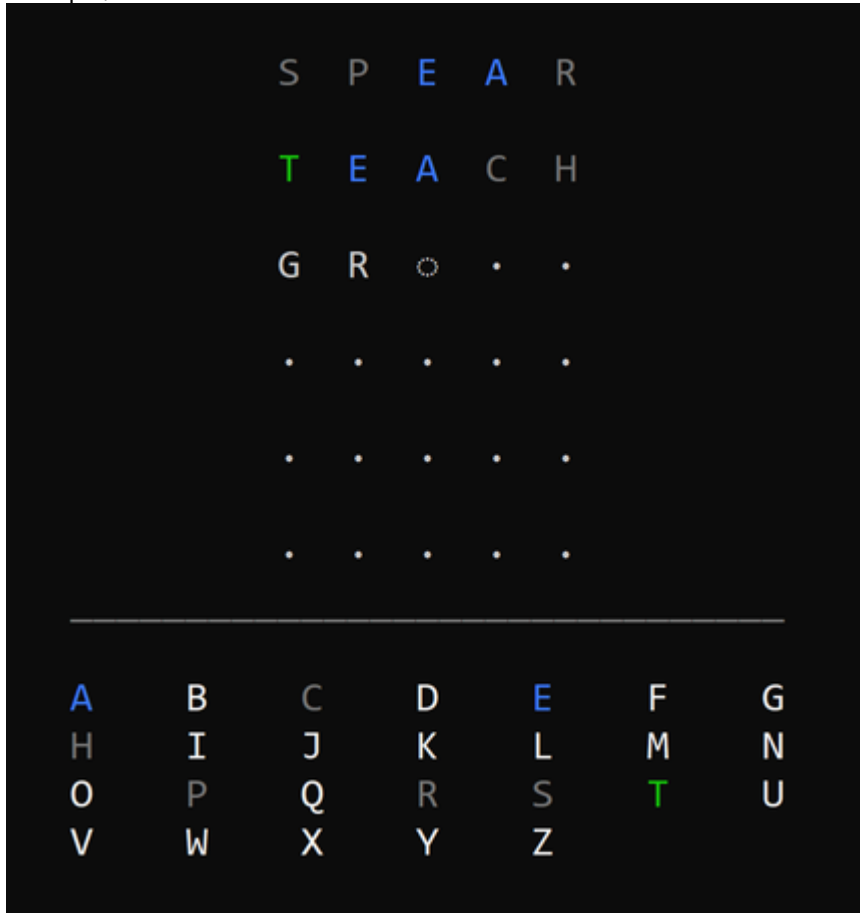
Figure out how you would write test routines (using MSTest) to test this code. Use the reference code as a starting point, but then modify it to make it more suitable for testing. In particular, we want to check at least the following:

- Correct behavior when various keystrokes are sent
- Correct *coloring* of the guessed words, the 'keyboard' at the bottom etc.
- Correct handling of a correct guess (success) or 6 tries without a correct guess (failure)

Tips:

- Since your test should be deterministic, have a way to 'force' a particular word as the secret word (to be used only during testing)
- Simulate key-presses by calling `UpdateGameState`
- Since we cannot check the visual display printed on the console, have the option that `DisplayBoard` saves the board to a text file. The text file should include color annotations. For

example, assume the screen looks like this:



By using some annotations for colors (like [E] for blue, {T} for green, (S) for gray, one could output this into a text file like this:

```
(S)(P)[E][A](R)
{T}[E][A](C)(H)
G R * . .
. . . . .
. . . . .
. . . . .

[A] B (C) D (E) F G
(H) I J K L M N
O (P) Q (R)(S){T} U
V W X Y Z
```

Then, the text files can be compared in the test. This way, we automate the testing that the correct text is being output with the correct colors

- To compare two text files, one could use logic like this:

```
bool CheckTextFilesEqual (string f1, string f2) {
    if (File.ReadAllText (f1) == File.ReadAllText (f2)) return true;
    var p = System.Diagnostics.Process.Start ("winmerge.exe", $"\"{f1}\"
\"{f2}\"");
    p.WaitForExit ();
}
```

```
        return false;
    }
```

This checks if the files are equal, and if they are not, it starts the WinMerge program to show you a visual diff between the two files (which is very useful during testing). For this to work, install **WinMerge** and add that installation folder to your system path.